

Extending QMIX with Simple Techniques from Single-Agent Reinforcement Learning

Lucheng Wang

201769571

1. Statement of Ethical Compliance

This project involves data and participant categories A0. I guarantee that I will obey the ethical guidelines throughout the project.

2. Project Description

This project is a research focus project, which aims to extend QMIX, a popular cooperative multi-agent reinforcement learning algorithm that performs competitively on a complex set of StarCraft II micromanagement tasks [1], with current fundamental methods from single-agent reinforcement learning. DQN (deep Q-network) is a classic and highly efficient single-agent reinforcement learning algorithm [2], and six extensions to DQN were proposed in Rainbow [3]. This project will primarily explore two of the six extensions: the Dueling Network and Multi-step Learning. Whether to explore the remaining four techniques will be evaluated according to the progress of the project and other pertinent considerations.

The performance of the optimized QMIX algorithm will be mainly examined on simple cooperative multi-agent tasks, such as the Predator-Prey tasks. The optimized QMIX will be compared and analyzed with the original QMIX regarding the performance, and the optimization method will be refined through iterative adjustments, ultimately producing a more satisfactory optimization strategy.

The project will be established on the PyMARL. It is a framework open-sourced by DeepMind, which supports the implementation of various MARL (multi-agent reinforcement learning) algorithms, offers essential components required for MARL, and can efficiently execute on computing clusters.

This project predominantly contributes to the improvement of cooperative multi-agent reinforcement learning by demonstrating the potential for transferring single-agent enhancements to multi-agent.

3. Aims & Requirements

This project aims to optimize the QMIX multi-agent reinforcement learning algorithm with single-agent reinforcement learning algorithm optimization techniques. By implementing these extensions, the project is expected to achieve these goals:

Aims

1. **QMIX Optimization:** Transfer DQN algorithm optimizations in Rainbow to QMIX separately. Priority will be given to implementing the Dueling Network and Multi-step Learning techniques. If time permits, additional methods for example Double Q-learning, Prioritized Replay, Distributional RL, and Noisy Net will be incorporated.

2. **Validating Algorithm Effectiveness in Complex Tasks:** Compare the performance of the optimized QMIX with the original version, analyze a general orientation of improvement, and apply iterative refinements to adjust the algorithm.

Requirements

Essential Requirements:

1. **QMIX Background Reading:** Fully understand the QMIX algorithm, familiarizing its strengths and weaknesses. Brainstorm optimization strategies.
2. **Dueling Network and Multi-step Learning Background Reading:** Study the benefits of the Dueling Network and Multi-step Learning techniques, exploring the potential interaction with traditional QMIX algorithm.
3. **Implementing Dueling Network and Multi-step Learning to QMIX:** Extend the QMIX algorithm individually using the Dueling Network and Multi-step Learning techniques. Dueling Network: divide the estimation of Q-values into a value function and an advantage function. Multi-step Learning: accumulate rewards over multiple time steps to calculate the target Q-value.
4. **Environment Configuration:** Set up the environment on the Barkla System.
5. **Experiment and Test:** Conduct baseline test and optimized algorithm experiment in the Predator-Prey tasks.
6. **Performance Evaluation:** Evaluate the performance of the optimized algorithm compared with the benchmark.
7. **Performance Comparison and Iterative Optimization:** Through performance comparison with the original QMIX algorithm, identify the strengths and weaknesses of the optimizations, and iteratively refine it for improvement.
8. **Review and Summary:** Review the entire research process and reflect on mistakes made during the study. Additionally, consolidate insights from optimizing the QMIX multi-agent algorithm and conclude the experiences in research abilities and project management.

Desirable Requirements:

1. **Further Implementing Other Techniques in Rainbow to QMIX:** Learn the theories within other techniques in Rainbow such as Double Q-learning, Prioritized Replay, Distributional RL, and Noisy Nets methods. Attempt to extend the QMIX algorithm with them.
2. **Evaluate the Performance on Additional Tasks:** Beyond the Predator-Prey tasks, choose additional multi-agent tasks to verify the general applicability of the optimization techniques.

4. Key Literature & Background Reading

With the advancements in deep learning techniques and computing resources, deep reinforcement learning has achieved remarkable success across various tasks, for example, Atari games [4], StarCraft [1], and robot control [5]. VDN and QMIX are famous and fundamental deep reinforcement learning algorithms that can solve factorizable MARL tasks [6]. The primary reason why choosing QMIX for this project is that it outperforms VDN in both absolute performance and learning speed [1]. More specifically, QMIX has the following enhancement upon VDN:

Mixing Network: QMIX changes the factorization of the joint Q-value function, substituting the linear summation of local Q-value in VDN to a non-linear combination of agent Q-value. QMIX ensures IGM (individual-global-maximum) by enforcing the monotonicity of the relationship between Q_{tot} and Q_a :

$$\frac{\partial Q_{tot}}{\partial Q_a} \geq 0, \forall a \in A. \quad (1)$$

This could guarantee the contribution of each agent is positive to the joint Q-value. Therefore, the global argmax applied to Q_{tot} produces the same outcome as performing individual argmax operations on each Q_a :

$$\operatorname{argmax}_u Q_{tot}(\tau, u) = \begin{pmatrix} \operatorname{argmax}_{u^1} Q_1(\tau^1, u^1) \\ \vdots \\ \operatorname{argmax}_{u^n} Q_n(\tau^n, u^n) \end{pmatrix}. \quad (2)$$

Utilization of a mixing network enables QMIX to adapt to the complicated cooperative relationship in multi-agent environments since it can capture the dependent connections between agents and ensure that every individual action optimizes the global strategy.

Hypernetwork: QMIX introduces a hypernetwork to calculate the parameters of the mixing network. Additionally, the hypernetwork uses the global state, allowing the network to adjust the weighting of each Q-value. The non-negative weights generated by the hypernetwork ensure the monotonicity constraint, maintaining consistency between each local optimal action and the global optimal action.

However, the monotonicity constraint decreases the Q-value expressiveness in QMIX, which might be over-restricted and affect performance in challenging environments [7]. In addition, the representational constraints on the joint action-value, introduced by QMIX, result in demonstrably inadequate exploration and suboptimality [8]. Thus, investigating methods to optimize QMIX in this project is essential.

The main focus of this research is on two techniques: Dueling Network and Multi-step Learning, which have been implemented in DQN single-agent algorithm and proved to be efficient.

Dueling Network: The Dueling Network architecture, which incorporates two distinct estimators: one concentrated on the state value function and another on the state-dependent action advantage function, has been proposed and proven to better policy evaluation, generalization, and stability [9].

Multi-step Learning: Multi-step Learning, also known as n-step learning, is distinct from one-step learning, not only relying on the current step reward but also accumulating rewards over the next n steps. The update formula is calculated through n-step TD (temporal difference). This approach augments robustness and learning performance [10].

The PyMARL framework, open-sourced by DeepMind, provides efficient parameter configuration functions and adaptability to diverse platforms. Furthermore, it enables the efficient implementation of various multi-agent algorithms and supports a wide range of experimental environments [11].

One of the major limitations of the QMIX algorithm is Q-value issues. Dueling Networks and Multi-step Learning have been shown to significantly enhance the robustness and expressiveness of Q-values [3]. Furthermore, QMIX has similar parts as DQN, such as the loss function and the method of computing the Q-value of each agent. These criteria make this combination promising.

The experiments of this project will be mainly executed on HPC (high-performance computing) within a limited research time frame. Therefore, framework compatibility and efficiency are also essential.

5. Development & Implementation Summary

Development Environment & Implementation Language

Development Environment:

PyMARL Framework: This project is based on multi-agent reinforcement learning, and the framework offers a robust system for implementing and testing multi-agent algorithms. There are three vital parts in this framework:

Module (src/module/mixer): Include the basic network structure of QMIX, where this project will mainly modify the file **qmix.py**.

Environment (src/envs): Responsible for defining and managing the interface with the environment. The Predator-Prey environment interface will be set here.

Configuration (src/config): Contains the configuration of algorithms and environments. The configuration file of QMIX is **qmix.yaml**. In addition, the configuration file of the Predator-Prey environment should be integrated into this framework.

Barkla HPC Cluster: Running complex multi-agent reinforcement learning experiments requires a device that can operate reliably and efficiently over long periods. Barkla is a high-performance computing cluster system of the University of Liverpool, where users can schedule jobs simultaneously.

Implement Language:

Python: Python was chosen as the primary programming language because the PyMARL framework is mainly developed in Python. Moreover, Python

provides a wide range of deep-learning libraries and a highly supportive development environment.

Project Component Breakdown

Core Algorithm Implementation: Combine Dueling Network or Multi-step Learning with QMIX algorithm. This is the core part of the project and the fundamental of the following components.

Agent Training Module: Set up the experimental environment on the Barkla Server. Then run numerous agent training tasks according to the set parameters of the implemented optimized QMIX algorithms.

Performance Evaluation and Analysis: Define specific evaluation criteria to compare the performance between the optimized algorithm and the original algorithm. Analyze performance differences and discover the underlying reason behind these gaps.

Iterative Optimization: Adjust the algorithm or parameter according to the analysis results. Record each adjustment and the outcomes of each experiment in detail, concluding patterns to determine a more efficient direction of further fine-tuning.

How the Components Cooperate: **Core Algorithm Implementation** establishes a solid fundamental for other components. **Agent Training Module** tests the algorithm in environments. **Performance Evaluation and Analysis** handle the training results and refine the algorithm. **Iterative Optimization** concludes a general fine-tuning orientation which helps further improvement and conducts a new round of optimization. The entire process can be likened to a loop, with each component tightly interconnected.

6. Data Sources

This project is a theoretical research focus project. All data required for this project is simulated data generated through test environments. It will not involve any open-source datasets, publicly available text, and APIs.

This section does not apply to this project

7. Testing & Evaluation

Test Environment

This project will primarily conduct experiments in the Predator-Prey environment. In this environment task, the multiple agents, the predators, are expected to cooperate to capture the prey. Once two predators simultaneously perform the capture action on one prey, then the capture is successful, and agents should be given positive rewards. Otherwise, the agents will receive the negative rewards. There are several parameters that could be modified, the grid size, observation range, and penalty parameter p .

Test & Experiment Workflow

1. **Environment Configuration:** Set up the environment for experiments with all necessary dependencies.
2. **Baseline Testing:** Conduct baseline tests for the original QMIX and establish a benchmark for further experiments.
3. **Optimized Algorithm Testing:** Test the modified QMIX algorithm in the same environment as baseline testing.
4. **Evaluation and Fine-tuning:** Compare the outcome to the benchmark and analyze potential orientations for further fine-tuning. The possibly modifiable parameters include the learning rate, batch size, n in n -step TD, regularization in the advantage function, and so on.
5. **Iterative Optimization:** Repeat steps 3 and 4 until gaining a desirable and promising result.
6. **Multitask testing:** Change the task or the parameters of the task and repeat steps 2, 3, 4, and 5 to obtain various results.

Evaluation Criteria

For the predator-prey task, common evaluation criteria consist of Test Return, Test Win Rate, Convergence Speed, and so forth. This project plans to generate line charts for each criterion across all tasks, to reflect the trends and gaps more intuitively. The goal of this project is to achieve an improvement of about 10% in 80% of the tasks.

8. Project Ethics & Human Participants

This project is a theoretical research focus project. All requirement analysis and evaluation in this project are completed independently by myself, with no other human participants. Thus, no additional ethical considerations are necessary for this project.

This section does not apply to this project

9. BCS Project Criteria

This project will attempt to meet the requirements of BCS (British Computer Society) and deliver an innovative and promising technique in this region.

1. **Practical and Analytical Skills:** This project is related to the QMIX reinforcement learning algorithm, which mainly uses Python as the programming language. It demands strong programming skills and a solid foundation of AI knowledge. The analysis aspect involves abilities to evaluate, analyze, and optimize algorithm performance.
2. **Innovative and Creativity:** This project will transfer optimization techniques from single-agent algorithms to multi-agent algorithms, modifying the parameters, formulas, and even network structures to achieve better performance. Additionally, more multi-agent tasks, including Predator-Prey tasks with different parameters and complicated StarCraft tasks, may be tested.

To better reach these targets, it is necessary to review extensive literature, understand new techniques, and try to implement them in this project.

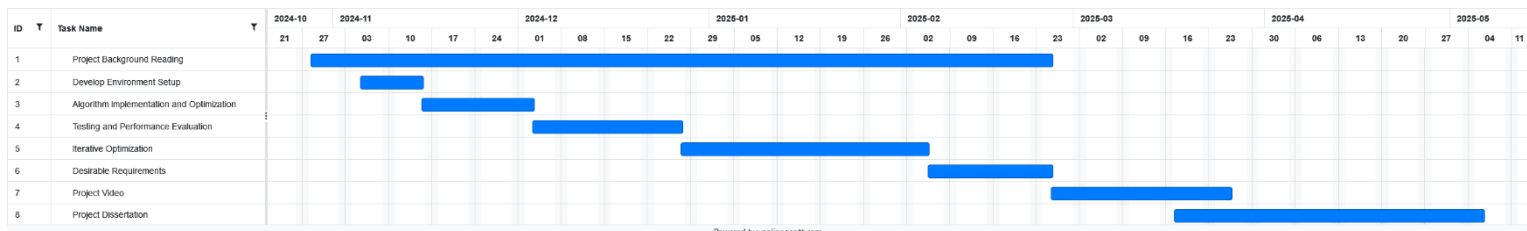
3. **Synthesis Information and Evaluation of Solution:** In this project, the testing and experimental part will repeat two sections: analyzing results and applying insights to further refine parameters or modify optimization methods. This cyclical process emphasizes the integration of information, ideas, and applications.
4. **Fulfilling a Broader Need:** Multi-agent reinforcement learning has a wide range of applications in the real world, including issues such as robotic collaboration. In this project, running efficiency optimization can decrease the latency while practical use and performance optimization guarantee greater robustness and accuracy for devices. Simultaneously, multi-agent optimization approaches can promote further research in this area and offer inspiration for research in other fields.
5. **Self-Management of Major Work:** The project is carefully separated into multiple components, with a Gantt chart to track the progress of tasks. The iteration of test, evaluation, and implementation phases also showcase the ability of efficient management and the capacity to progressively perfect complex tasks.
6. **Critical Self-Evaluation:** Summarize and reflect on the achievement, the challenge encountered, and the solution, at the end of each development stage. Record the consequence of each iteration to figure out the further optimizing direction. In the final report, review the overall approach and decisions to determine the potential improvement of the whole project.

10. UI/UX Mockup

This project is a theoretical research focus project. Therefore, there will not be any UI or UX elements.

This section does not apply to this project

11. Project Plan



12. Risks & Contingency Plans

Risks	Contingencies	Likelihood	Impact
Hardware Failure	Regular backup file from Barkla server to local. Ensure that primary work is done on reliable hardware.	Medium	High
Software Failure	Use the correct version of software. Manage code by using version control tools.	Medium	Medium
Runing out of time	Create a Gantt chart a with clear timeline and deadline. Track the progress of each task.	Low	High
Difficulty in Improving algorithm	Do more literature reviewing. Try adjusting parameters or optimizing with other strategies.	High	High
Illness	Allow sufficient time in advance.	High	Low
Insufficient computational resources	Attempt to train agents by using CPU instead of GPU.	High	Medium
Unfamiliar with PyMARL framework	Learn this framework in advance. Seek help from related forums.	Medium	Medium

13. Reference

- [1] T. Rashid, M. Samvelyan, C. S. De Witt, G. Farquhar, J. Foerster, and S. Whiteson, "Qmix: Monotonic value function factorisation for deep multi-agent reinforcement learning," *Proc. Int. Conf. Mach. Learn.*, pp. 4292–4301, 2018.
- [2] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [3] M. Hessel, J. Modayil, H. Van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar, and D. Silver, "Rainbow: Combining improvements in deep reinforcement learning," *Proc. Thirty-Second AAAI Conf. Artif. Intell.*, pp. 3215–3222, 2017.
- [4] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

- [5] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015. Available: <https://arxiv.org/abs/1509.02971>.
- [6] K. Son, D. Kim, W. J. Kang, D. E. Hostallero, and Y. Yi, "QTRAN: Learning to Factorize with Transformation for Cooperative Multi-Agent Reinforcement Learning," *arXiv preprint arXiv:1905.05408*, 2019. Available: <https://arxiv.org/abs/1905.05408>.
- [7] T. Rashid, G. Farquhar, B. Peng, and S. Whiteson, "Weighted QMIX: Expanding Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning," *arXiv preprint arXiv:2006.10800*, 2020. Available: <https://arxiv.org/abs/2006.10800>.
- [8] A. Mahajan, T. Rashid, M. Samvelyan, and S. Whiteson, "MAVEN: Multi-Agent Variational Exploration," *arXiv preprint arXiv:1910.07483*, 2020. Available: <https://arxiv.org/abs/1910.07483>.
- [9] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling Network Architectures for Deep Reinforcement Learning," *arXiv preprint arXiv:1511.06581*, 2016. Available: <https://arxiv.org/abs/1511.06581>.
- [10] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA: MIT Press, 1998.
- [11] M. Samvelyan, T. Rashid, C. Schroeder de Witt, G. Farquhar, N. Nardelli, T. G. J. Rudner, C.-M. Hung, P. H. S. Torr, J. Foerster, and S. Whiteson, "The StarCraft Multi-Agent Challenge," *CoRR*, vol. abs/1902.04043, 2019.